

torch.functional.cdist#

<https://pytorch.org/docs/stable/generated/torch.functional.cdist.html>

Description:

Computes batched the p-norm distance between each pair of the two collections of row vectors. `x1` (Tensor) – input tensor where the last two dimensions represent the points and the feature dimension respectively. The shape can be $D_1 \times D_2 \times \dots \times D_n \times P \times M$, where P is the number of points and M is the feature dimension. `x2` (Tensor) – input tensor where the last two dimensions also represent the points and the feature dimension respectively. The shape can be $D_1' \times D_2' \times \dots \times D_m' \times R \times M$, where R is the number of points and M is the feature dimension, which should match the feature dimension of `x1`. `p` (float) – p value for the p-norm distance to calculate between each vector pair $\in [0, \infty]$. `compute_mode` (str) – ‘use_mm_for_euclid_dist_if_necessary’ - will use matrix multiplication approach to calculate euclidean distance ($p = 2$) if $P > 25$ or $R > 25$ ‘use_mm_for_euclid_dist’ - will always use matrix multiplication approach to calculate euclidean distance ($p = 2$) ‘do_not_use_mm_for_euclid_dist’ - will never use matrix multiplication

Function Signature

```
torch.functional.cdist(x1, x2, p=2.0,  
compute_mode='use_mm_for_euclid_dist_if_necessary')[source]#
```

Parameters

Return type

Returns:

Tensor

Code Examples

```
>>> a = torch.tensor([[0.9041, 0.0196], [-0.3108, -2.4423],  
                      [-0.4821, 1.059]])  
>>> a  
tensor([[ 0.9041,  0.0196],  
        [-0.3108, -2.4423],  
        [-0.4821,  1.0590]])  
>>> b = torch.tensor([[-2.1763, -0.4713], [-0.6986, 1.3702]])  
>>> b  
tensor([[-2.1763, -0.4713],  
        [-0.6986,  1.3702]])  
>>> torch.cdist(a, b, p=2)  
tensor([[3.1193, 2.0959],  
        [2.7138, 3.8322],  
        [2.2830, 0.3791]])
```

Detailed Documentation

Documentation

Computes batched the p-norm distance between each pair of the two collections of row vectors.

x_1 (Tensor) – input tensor where the last two dimensions represent the points and the feature dimension respectively. The shape can be $D_1 \times D_2 \times \cdots \times D_n \times P \times MD_1 \times D_2 \times \cdots \times D_n \times P \times M$, where PPP is the number of points and MMM is the feature dimension.

x_2 (Tensor) – input tensor where the last two dimensions also represent the points and the feature dimension respectively. The shape can be $D_1' \times D_2' \times \cdots \times D_m' \times R \times MD_1' \times D_2' \times \cdots \times D_m' \times R \times M$, where R is the number of points and M is the feature dimension, which should match the feature dimension of x_1 .

p (float) – p value for the p -norm distance to calculate between each vector pair $\in [0, \infty] \cup [0, \infty]$.

`compute_mode` (str) – ‘use_mm_for_euclid_dist_if_necessary’ - will use matrix multiplication approach to calculate euclidean distance ($p = 2$) if $P > 25$ or $R > 25$ ‘use_mm_for_euclid_dist’ - will always use matrix multiplication approach to calculate euclidean distance ($p = 2$) ‘donot_use_mm_for_euclid_dist’ - will never use matrix multiplication approach to calculate euclidean distance ($p = 2$) Default: use_mm_for_euclid_dist_if_necessary.

If x_1 has shape $B \times P \times MB \times P \times M$ and x_2 has shape $B \times R \times MB \times R \times M$ then the output will have shape $B \times P \times RB \times P \times R$.

This function is equivalent to `scipy.spatial.distance.cdist(input, 'minkowski', p=p)` if $p \in (0, \infty)$. When $p=0$ it is equivalent to `scipy.spatial.distance.cdist(input, 'hamming') * M`. When $p=\infty$, the closest scipy function is `scipy.spatial.distance.cdist(xn, lambda x, y: np.abs(x - y).max())`.